

Sandbox:

- Constraints: YARA-Regeln + ML, Monitoring, Kosten jährlich

Sandbox = Isolation + Beobachtung

Unbekannte Software → Sandbox → beobachten: Prozesse, Dateien, Registry, Netzwerk, Speicher, Systemaufrufe

„Lass ihn möglichst realistisch arbeiten, aber in einer kontrollierten Umgebung, und zeichne alles auf.“

Windows-Sandboxing:

Restricted Tokens: Windows erstellt ein eingeschränktes Access Token und entfernt Rechte bzw. Gruppen. Zugriffe werden dann mit diesem reduzierten Token geprüft.

Integrity Levels: Prozesse und Objekte bekommen eine Stufe wie Low, Medium, High, System. Ein Prozess mit niedrigerer Stufe darf höhere Objekte nicht verändern.

AppContainer: Der Prozess bekommt ein eigenes eingeschränktes Token mit AppContainer-SID und Capabilities. Nur ausdrücklich erlaubte Zugriffe auf Dateien, Registry, Netzwerk oder Geräte sind möglich.

Job Objects: Mehrere Prozesse werden in einem Kernel-Objekt zusammengefasst. Windows kann dafür CPU, RAM, Prozessanzahl und Child-Prozesse begrenzen.

ACLs: Dateien, Registry und andere Objekte besitzen Zugriffslisten. Windows vergleicht diese mit dem Access Token und erlaubt oder blockiert den Zugriff.

Process Mitigations: Erschweren Speicherfehler und Exploits im Prozess durch Aktivierung von DEP, ASLR, CFG.

WDAC: Windows prüft Code anhand von Signatur, Publisher oder Hash gegen eine Policy. Nicht erlaubte Programme, DLLs oder Treiber werden blockiert. Application Allowlisting / Code Integrity.

Container: Eigenes Dateisystem, Netzwerk, Prozesssicht. Aber System Calls gehen zum gleichen Host-Kernel.

VM simuliert einen eigenen Computer. Eigener Kernel, Prozesse, RAM. Aber die Hardware ist virtuell: CPU, RAM, Disk.

Hypervisor erstellt und verwaltet die VM. Der Hypervisor ist die Virtualisierungsschicht. Hypervisor sitzt außerhalb der VM.

Überwachung in VM: Agent / Hooks: Malware → CreateProcess() → [Sandbox Hook] → Windows

Sandbox Evasion: Malware erkennt Sandbox (Agent / Hooks) → nichts Böses tun.

User-Mode Hooks: Eine normale Anwendung läuft im User Mode. Dort gibt es APIs wie: CreateFile, CreateProcess, RegOpenKey, WinHttpSendRequest. Diese sind einfach zu hooken. Fortgeschrittene Malware kann sie umgehen.

Application

User Mode

Kernel Mode

Hardware

Malware

↓

WinAPI

↓

ntdll

↓

Syscall

↓

Kernel

Malware

↓

direkter Syscall

↓

Kernel

Windows grob: Problem: **Direct Syscalls:** Normal: Fortgeschritten:

Deshalb ist interessant: Werden nur Windows APIs überwacht oder auch niedrigere System Calls?

Hypervisor-basiertes Monitoring: Anstatt viel Code in Windows hineinzubauen, versucht man, von außen zu beobachten. Vorteile: weniger sichtbare Analyseartefakte + tieferer Blick auf Speicher / CPU / Kernel

VMI (Virtual Machine Introspection) = Der Hypervisor sieht zunächst nur technische Rohdaten. z.B.: 0x7ffab23000
aber nicht automatisch: chrome.exe Sandbox braucht deshalb Wissen über die internen Datenstrukturen des OS. Dann kann sie beispielsweise rekonstruieren: Dieser Speicherbereich → gehört zu Prozess XYZ → PID 1234 → malware.exe

Anti-Evasion: „Malware soll nicht künstlich wirken.“ Realistischer Umgebung: 8 CPU Threads, 16 GB RAM, 500 GB SSD, Browser History, Dokumente, Software, Benutzeraktivität, Mausbewegungen, realistische Hostnamen/Netzwerkaktivität. Zusätzlich reduziere Spuren des Hypervisors oder der Überwachungssoftware.

Sleep Evasion: Malware: Sleep(2 Stunden). Sandbox-Techniken können versuchen Zeitquellen zu manipulieren: CPU Timer, System Clock, Network Time, Performance Counter. Malware kann wiederum Zeitquellen vergleichen. Damit entsteht ein Katz-und-Maus-Spiel.

User Interaction Evasion: Manche Malware startet erst, wenn ein echter Benutzer aktiv ist. Z. B.: Maus bewegt? Dokument geöffnet? Button geklickt? Browser verwendet? Captcha gelöst? Deshalb simulieren Sandboxes Benutzeraktivität. Bei interaktiven Sandboxes kann auch ein Analyst selbst klicken.

Dynamische Analyse: Die Datei wird tatsächlich ausgeführt.

Statische Analyse: Datei untersuchen, ohne sie auszuführen.

Memory Analysis: Speicher untersuchen. Wichtig, da Malware zunächst verschlüsselt oder gepackt ist.



Auf Disk: verschlüsselt/gepackt Im Speicher: echter Code erscheint im RAM Z. B.: Memory Dumps, Injected Code, Shellcode, C2-Adressen

Process Injection: Malware kann Code in einen legitimen Prozess einschleusen. Z. B.: malware.exe → öffnet explorer.exe → schreibt Code hinein → startet Code dort.

Netzwerkaktivität erscheint: explorer.exe → evil.example

Was eigentlich passiert: malware.exe → evil.example

Network Analysis. Interessante Daten: DNS Queries, IP-Adressen, Domains, URLs, HTTP Requests, TLS-Verbindungen, PCAP (Paketaufzeichnung), C2-Muster.

Problem: TLS-Verschlüsselung. Einige Sandboxes versuchen deshalb, TLS-Verbindungen zu entschlüsseln oder auf anderer Ebene mitzuschneiden.

Emulation: Die Hardware wird per Software nachgebaut. Beispiel: Eine x86-CPU wird nicht direkt benutzt, sondern ihre Befehle werden von einem Emulator interpretiert oder übersetzt. Langsamer aber sehr tiefe Analyse, z. B. einzelne CPU-Befehle, Speicherzugriffe oder Kontrollfluss.

Bare Metal: Malware läuft auf echter physischer Hardware. System muss nach jeder Analyse zuverlässig zurückgesetzt werden. Komplex und teuer.

Cloud: Die Infrastruktur läuft beim Anbieter.

YARA: eine Sprache zum Erkennen von Dateien oder Daten anhand von Mustern. Bspw.: Wenn Datei enthält: "evil_config" + bestimmte Byte-Sequenz + PE-Datei → möglicherweise Malware X

Sigma = „YARA für Log-/Event-Muster“

Suricata = „YARA für Netzwerkverkehr“

Frage	Warum wichtig?
Wo läuft das Sample?	VM, Emulation, Bare Metal
Wie wird beobachtet?	Agent, Hooks, Hypervisor, VMI
Wie tief wird beobachtet?	API, Syscalls, Kernel, Memory
Wie gut ist Anti-Evasion?	Erkennt Malware die Umgebung?
Was wird analysiert?	Static, Dynamic, Memory, Network
Wie können wir eigene Regeln nutzen?	YARA, Sigma, Suricata
Wie automatisierbar ist es?	API, Batch, Pipelines
Was passiert mit Ergebnissen?	Reports, IOCs, Threat Intelligence

Linux-Sandboxing:

- **chroot** ist eine alte Unix/Linux-Technik. Sie verändert hauptsächlich, welches Verzeichnis ein Prozess als / sieht.

- **Linux-Sandboxing** = Namespaces bestimmt: „Was kann der Prozess sehen?“

```
Ich bin PID 1.  
Ich habe mein eigenes Netzwerk.  
Ich sehe nur mein Dateisystem.  
Ich habe eigene Benutzer.
```

Ein Prozess kann glauben: Obwohl alles auf demselben Linux-Kernel läuft.

Das ist eine der Grundlagen von Docker und anderen Container-Technologien.

- **cgroups** bestimmen: „Wie viele Ressourcen darf der Prozess benutzen?“

- **seccomp-bpf** ist eine Linux-Technik, mit der man System Calls filtern kann.